ScienceDirect

Computers & Security

Volume 126, March 2023, 103068

VERI: A Large-scale Open-Source Components Vulnerability Detection in IoT Firmware

Yiran Cheng [a b], Shouguo Yang [a b], Zhe Lang [a b], Zhiqiang Shi [a b], Limin Sun [a b]

Show more

https://doi.org/10.1016/j.cose.2022.103068

Abstract

IoT device manufacturers integrate open-source components (OSCs) to serve necessary and common functions for facilitating firmware development. However, outdated versions of OSC conceal N-day vulnerabilities and continue to function on IoT devices. The security risks can be predicted once we can identify the OSC versions employed in the firmware. Existing works make attempts at OSC version identification but fail to perform vulnerability detection on a large-scale IoT firmware due to i) unsuitable version identification method for IoT firmware scenario. ii) the lack of a large-scale version-vulnerability relation database. To this end, we propose a system VERI for large-scale vulnerability detection based on lightweight version identification. First, for OSC version identification, VERI leverages symbolic execution with static analysis to identify exact OSC versions even though there are many version-like strings in OSC. Second, VERI employs a deep learning-based method to extract OSC names and vulnerable version ranges from vulnerability descriptions, constructs and maintains an OSC version-vulnerability relation database to serve the vulnerability detection. Finally, VERI polls the relation database to confirm the N-day security risk of the OSC with identified version. The evaluation results show that VERI achieves 96.43% accuracy with high efficiency in OSC version identification. Meanwhile, the deep learning model accurately extracts the OSC names and versions from vulnerability descriptions dataset with 97.19% precision and 96.56% recall. Based on the model, we build a large-scale version-vulnerability relation database. Furthermore, we utilize VERI to conduct a large-scale analysis on 28,890 firmware and find 38,654 vulnerable OSCs with 266,109N-day vulnerabilities, most of which are with high risks. From the detection results, we find that after the official patch for the vulnerability is released, manufacturers delay an average of 473 days to patch the firmware.

Introduction

With the explosion of the Internet-of-Things (IoT), the number of embedded devices, from appliances to smart homes and personal gadgets, has proliferated dramatically. While the benefits of IoT devices can be observed everywhere, their inherent vulnerabilities do create new security risks(Alrawi et al., 2019). These vulnerabilities leave devices open to cyberattacks, which can disrupt industries and economies in dangerous ways. For example, the Mirai botnet compromised millions of IoT devices in late 2016 and overwhelmed targets with massive distributed denial-of-service (DDoS) attacks(Antonakakis et al., 2017).

For facilitating development, IoT device manufacturers use open-source component (OSC) to serve necessary and common functions running on the firmware. From perspective of component, firmware is an integrated component package programmed on IoT device. However, vulnerabilities in the OSC can be introduced into IoT devices and make them vulnerable. For instance, Heartbleed compromised millions of devices and leveraged them in buffer over-read attacks to leak sensitive information, which was reported in OpenSSL (Cve-2014-0160, Heartbleed - home, openssl - home).

To address these security issues faced by IoT devices, manufacturers provide new firmware releases for updates, including patches for known OSC vulnerabilities. However, due to the various types of fragmented devices, it is difficult for manufacturers to generate patches timely for each type of device(He, Zou, Sun, Liu, Xu, Wang, Shen, Wang, Li, 2022, Niesler, Surminski, Davi, 2021). Even if patched firmware is released, end users may neglect to update the firmware to mitigate the vulnerabilities. As a result, attackers conduct N-day vulnerability attacks, which are made easier by leveraging patch information(Wang et al., 2019). A 2021 audit report from Synopsys (Bla,2021) mentioned that more than 65% of manufacturer-issued firmware contains recurring OSC vulnerabilities, many of which were disclosed years ago.

Because N-day vulnerabilities exist in specific OSC versions, identifying the OSC versions allows the security risk to be predicted. Many binary version identification approaches have been proposed(Almanee, Payer, Garcia, Duan, Bijlani, Xu, Kim, Lee, 2017, Zhan, Fan, Chen, We, Liu, Luo, Liu, 2021). However, they present their own set of limitations when applied to OSC version identification in IoT firmware. i) In firmware scenario, OSCs in different architectures and compiler environment can result in significantly different control-flow-related and opcode-related features. However, ATVHUNTER(Zhan et al., 2021) extracts the Control-Flow Graph (CFG) and opcode sequences as the features to identify the version. The application of these features leads to a significant performance loss. ii) OSSPolice and ATVHUNTER(Duan, Bijlani, Xu, Kim, Lee, 2017, Zhan, Fan, Chen, We, Liu, Luo, Liu, 2021) aim to identify the third-party native library and the librarys version in Android apps. These tools are based on Android's library dependency relations, which cannot meet our research scenarios for IoT firmware and their feature extraction methods are incompatible with IoT firmware.

Once the versions of OSCs are identified, the security risk can be predicted by querying the vulnerability databases such as National Vulnerability Database (NVD)(NVD,2021). The Common Platform Enumeration (CPE) format is used by NVD to store information about vulnerable versions. However, the CPEs in certain CVE are missing or inconsistent (as shown in Section2.1) with the actual vulnerable version range, which may cause major delays in patching. We examined all CVEs from 2016 to 2018, there are 271 CPEs missing partial or even total vulnerable versions which include frequently-used OSCs.

We summarize the challenges of OSC vulnerability detection in IoT firmware as follows:

C1: Multiple version-like strings exist in the same OSC. Plenty of OSCs include multiple string literals which "look like" real version string (e.g., strings 1.0.0, 1.0.1, **1.0. 1*d***, and **1.0. 2*g*** contained in OpenSSL 1.0.2g at the same time). Therefore, simply using regular-expression matching will not find the real version string to identify the OSC version.

C2: Dynamic method is not scalable in IoT scenario. We assume that running the OSC with specific parameter can produce the OSC version. However, due to the diverse CPU architectures and peripherals of IoT devices, these OSCs are very difficult to run without the devices. Even though QEMU(QEM,2021) can conduct simple emulation of OSCs, it still cannot perform on a large scale(Zhengetal., 2019).

C3: Constructing version-vulnerability relation database. Due to the inconsistency between the CPE and the actual vulnerable version range (as described in Section2.1), there is a need to construct a precise version-vulnerability relation database. To this end, we need a method to automatically and accurately extract vulnerable versions from the vulnerability reports.

To address these challenges, we propose VERI, an N-day vulnerability detection system for large-scale IoT firmware OSCs. VERI leverages lightweight symbolic execution with static analysis to run OSC and identify its version (to solve the challenge **C2**). Specifically, it first recovers the whole control flow of the target OSC which consists of a Control-Flow Graph (CFG) and a Call Graph (CG). Next, it locates the entry point and the version-output point to execute the OSC between these two points. To this end, VERI determines the start block of *main* function with its input arguments as entry point, and it collects the version-like string literals and locates the basic blocks which read them as the *candidate version point*. Then it conducts CFG pruning with Depth First Search (DFS) algorithm and static taint analysis to find possible feasible paths to the version point. Finally, it leverages an input-driven symbolic execution to remove infeasible paths ended with wrong version-output point (to solve challenge **C1**), and output the real version string.

VERI maintains an OSC version-vulnerability relation database to check if the OSC version is vulnerable. Since vulnerability description text is unstructured, we utilize a novel entity-relation extraction model, which learns the patterns from the description structures to recognize OSC version range (to solve challenge **C3**). More specifically, given a vulnerability description, we consider the vulnerable version range extraction as two tasks: (i) locating the entities (OSC names and versions) and (ii) extracting relations between the name and version. To consider the information interaction between the two tasks, we take a joint neural network model to perform end-to-end relations extraction. Finally, we convert the relations into unified version range and construct the version-vulnerability relation database.

We implement VERI and evaluate it in OSC version identification and entity-relation joint extraction model, respectively. To evaluate OSC version identification in different compilation settings, consisting of different instruction set architectures (ISA) and optimization levels. We compile open-source projects in different ISAs to build a Cross-ISA dataset with 1044 OSCs. VERI

achieves 96.5% accuracy on the dataset for the version identification task. Besides, we build a Cross-optimization level dataset with 1252 OSC binaries and VERI achieves 96.6% accuracy. To evaluate the entity-relation joint extraction model, we collect a large dataset from NVD which covers 23,410 vulnerabilities and their descriptions. Among them, we manually label 3822 descriptions for model training, validating and testing. We show our model achieves high accuracy with 97.19% precision and 96.56% recall in OSC entity recognition, 94.29% precision and 91.86% recall in vulnerable version relation extraction.

We also conduct a large-scale OSC version identification on real-world firmware. Combined with the version-vulnerability relation database, we detect 38,654 vulnerable OSCs in 28,890 firmware, and each Mb binary takes a very short time of 0.572s on average. Furthermore, We gain an interesting insight into firmware security, which is during the firmware iteration, manufacturers do not update vulnerable versions in time for OSCs with 473 outdated days on average.

The main contributions of this work are as follows:

- We construct a large-scale IoT firmware dataset including 28,890 IoT firmware with 31 kinds from 46 IoT device manufacturers, which is representative to understand IoT security.
- We propose a novel system VERI for N-day vulnerability detection in IoT firmware. VERI leverages a lightweight symbolic execution with static analysis for OSC version identification and introduces an end-to-end neural network model to extract the vulnerable version range from unstructured vulnerability descriptions. Both show high accuracy.
- VERI reports the discovery of 38,654 vulnerable OSCs in 24,845 firmware, with 524 distinct CVEs detected. According to the results, we reveal the exploit type and severity of the vulnerabilities. We conduct further analysis on the firmware and find that OSC was outdated by 473 days on average despite the firmware having been published new release.

Section snippets

Background

This sectionaims to explain our motivations and the assumption.

We illustrate our insights for OSC version identification using a real-world OSC *tcpdump*. *Tcpdump* is a freely available, powerful packet analyzer(tcp,2022) which is widely used in firmware. We compiled *tcpdump* 4.9.0 source code (with compiler gcc 4.7.1 and optimization option O0) to get the OSC binary and take the partial code in Listing as motivation example. OSCs typically contain their version string literals, which are …

Overview

In this section, we describe the high-level architecture of VERI for detecting N-day security risk of OSC in firmware. The overview of our system VERI is shown in Fig.2, which consists of three parts. VERI's input is an entire firmware image and output is the vulnerable OSCs of firmware and its N-day security risks (vulnerabilities).

OSC Version Identification (part ①). VERI identifies OSC version automatically using a lightweight symbolic execution with static analysis. We unpack the firmware …

OSC version identification

This sectionpresents our design and implementation of the OSC version identification method. …

Vulnerable version range extraction

As shown in Fig.5, the input of our model is a sequence of words (tokens) from vulnerability description, which are then represented as word vectors through word embeddings. Then we combine the bidirectional sequential LSTM (BILSTM) with the self-attention to extract a more complex representation for each word with context. The softmax layer produces the outputs for entity recognition task: (i) entity labels (e.g., the corresponding output of token *OpenSSL* is B-COM, B denotes the first token …

Evaluation

In this evaluation, we aim to answer following research questions:

RQ1: What is the accuracy of VERI for OSC version identification?

RQ2: What is the quality of the vulnerable version range extraction model in VERI for constructing database?

RQ3: How practical is VERI to detect N-day vulnerabilities in large-scale firmware? What about the outdatedness of the OSC in firmware?

We implement VERI in Python with 7038 lines of code. All the evaluations are conducted at ubuntu server with 56 cores CPU of …

Discussion

In this section, we discuss the limitations of VERI and future research work.

Firmware Complexity. The implementation of our scheme depends on the unpacking of large-scale firmware. The IoT firmware is made of multiple components and has different filesystems. Linux-based system is the most frequently encountered embedded operating system(Costinetal., 2014), our firmware processing method can unpack it well. However, blob firmware is a large, single-binary embedded OS compiled from components …

Information extraction from vulnerability reports

A large number of vulnerabilities require automated methods to assist in analyzing vulnerability reports. Dongetal.(2019) utilized a pipeline model (VIEM) to extract structured information from reports of multiple vulnerability databases to examine the information consistency. VIEM has some limitations: i) Pipeline model treats entity recognition and relation extraction as two separate tasks, ignoring the interactions between the two tasks. ii) VIEM cannot directly extract the relation …

Conclusion

In this paper, we propose VERI, an automatic N-day vulnerability detection system, which precisely identifies the OSC version, extracts vulnerable version range by an entity-relation joint extraction model and maintains a database. Evaluation results show that the version identification method is accurate and effective, meanwhile, the joint extraction model performs well in extracting version range information. Based on application results, VERI is able to conduct large-scale vulnerability …

CRediT authorship contribution statement

Yiran Cheng: Conceptualization, Methodology, Formal analysis, Writing – original draft. **Shouguo Yang:** Data curation, Formal analysis, Writing – review & editing. **Zhe Lang:** Data curation, Validation. **Zhiqiang Shi:** Conceptualization, Resources, Writing – review & editing. **Limin Sun:** Supervision, Writing – review & editing. …

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper. …

Acknowledgement

This research has been partially funded by the Nature Science Associate Foundation of China under Grants No. U1766215. …

Yiran Cheng: Ph.D. student from the Institute of Information Engineering, Chinese Academy of Sciences, China. Her research interests include IoT security and program analysis. …

…

...

References (55)

X. Li *et al.*
A mining approach to obtain the software vulnerability characteristics
2017 Fifth International Conference on Advanced Cloud and Big Data (CBD)(2017)

S. Zheng *et al.*
Joint entity and relation extraction based on a hybrid neural network
Neurocomputing (2017)

angr - home. https://www.angr.io/. (Accessed on...

binwalk - home. https://www.github.com/ReFirmLabs/binwalk. (Accessed on...

Blackduck - home. https://www.synopsys.com/software-integrity.html. (Accessed on...

bzip2 - home. https://www.man7.org/linux/man-pages/man3/getopt.3.html. (Accessed on...

Cve-2014-0160. https://nvd.nist.gov/vuln/detail/cve-2014-0160. (Accessed on...

Heartbleed - home. https://heartbleed.com/. (Accessed on...

Ida pro - home. https://www.hex-rays.com/. (Accessed on...

Nvd - home. https://nvd.nist.gov/. (Accessed on...

View more references

Cited by (8)

A review on TPL compliance and vulnerability spread: detection approaches and defense tactics in open-source contexts ↗
2026, Proceedings of SPIE the International Society for Optical Engineering

Security risk assessment of IoT firmware binaries based on a knowledge graph ↗
2026, Proceedings of 2026 6th International Conference on Computer Network Security and Software Engineering Cnsse 2026

Static Code Analysis for IoT Security: A Systematic Literature Review ↗
2025, ACM Computing Surveys

A Literature Review on Security in the Internet of Things: Identifying and Analysing Critical Categories ↗
2025, Computers

Vulnerability detection in Java source code using a quantum convolutional neural network with self-attentive pooling, deep sequence, and graph-based hybrid feature extraction ↗
2024, Scientific Reports

Comprehensive vulnerability aspect extraction ↗
2024, Applied Intelligence

View all citing articles on Scopus ↗

Yiran Cheng: Ph.D. student from the Institute of Information Engineering, Chinese Academy of Sciences, China. Her research interests include IoT security and program analysis.

Shouguo Yang: Ph.D. student from the Institute of Information Engineering, Chinese Academy of Sciences, China. His research interests include binary code similarity detection, patch detection, and vulnerability detection based on static analysis.

Zhe Lang: Ph.D. student from the Institute of Information Engineering, Chinese Academy of Sciences, China. His research interests include software security and program analysis.

Zhiqiang Shi: Ph.D. professorate senior engineer, Ph.D. supervisor from the Institute of Information Engineering, Chinese Academy of Sciences, China. His main research interests include network system and ICS security.

Limin Sun: Ph.D. professor, Ph.D. supervisor from the Institute of Information Engineering, Chinese Academy of Sciences, China. His main research interests include ICS security and IoT security.

View full text